System Design Document: Unified Service Scheduler

Keyloop Technical Assessment, Scenario A (Ownership domain)

Author: Ramdayal Patel | Implemented layer: Backend (C# / ASP.NET Core) | Frontend: stubbed with an OpenAPI spec and cURL examples

1. Overview

Problem. Dealerships still book service appointments by hand. The scheduler must confirm a booking only when a ServiceBay and a qualified Technician are both free for the entire service duration, then store a confirmed Appointment that links the customer, vehicle, technician and bay.

In scope: booking, real-time availability check, persistent Appointment record, retrieval, cancellation, slot search.

Out of scope (stubbed or designed only): customer-facing UI, identity provider, notifications, payments.

Design goals:

- Correctness first: a bay or technician can never be double-booked, even under concurrent requests.
- Clear, machine-readable failures (why a slot was refused).
- Observable in production (logs, metrics, traces).
- Easy to run: one `docker compose up` for the API and the database.

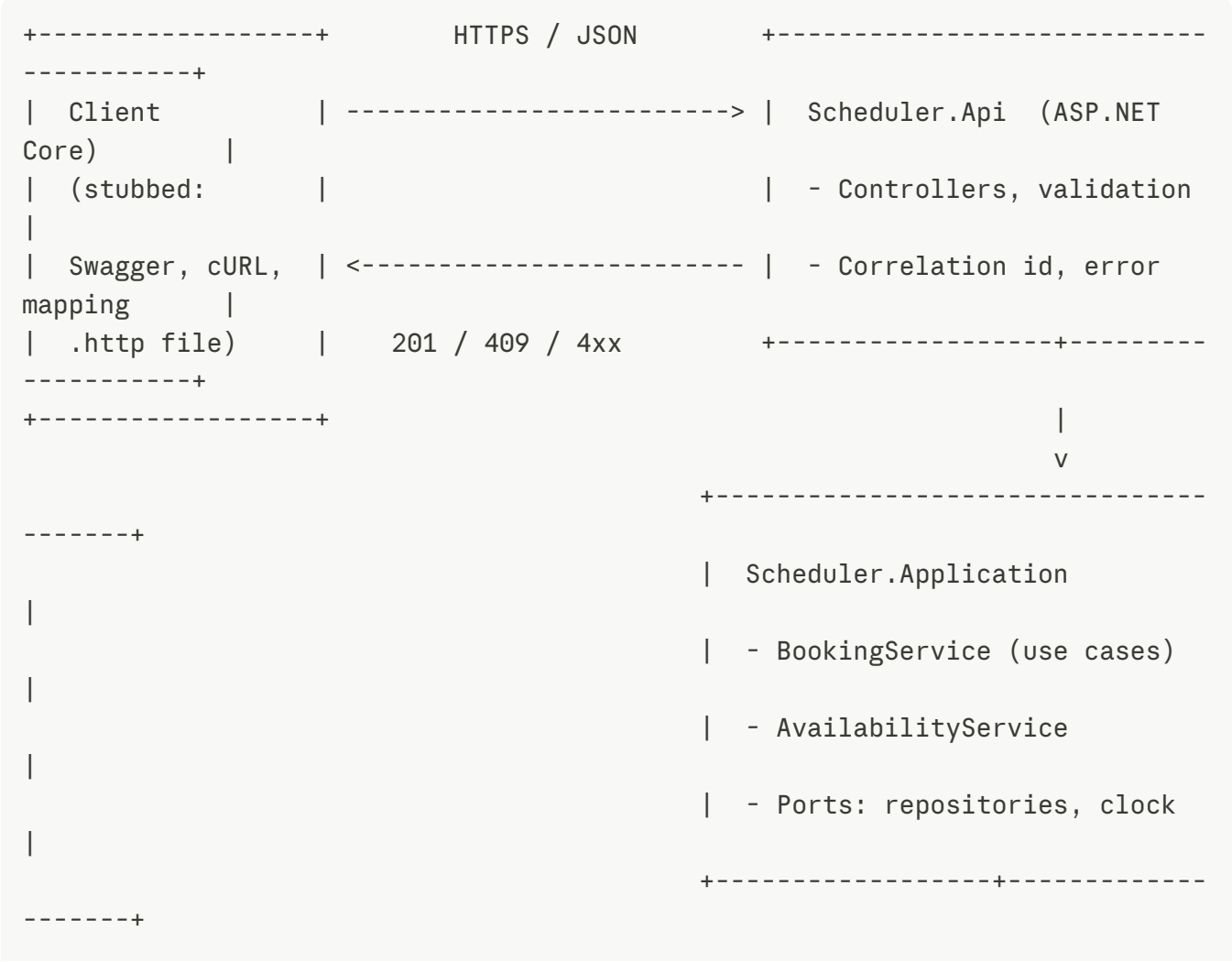
2. Assumptions

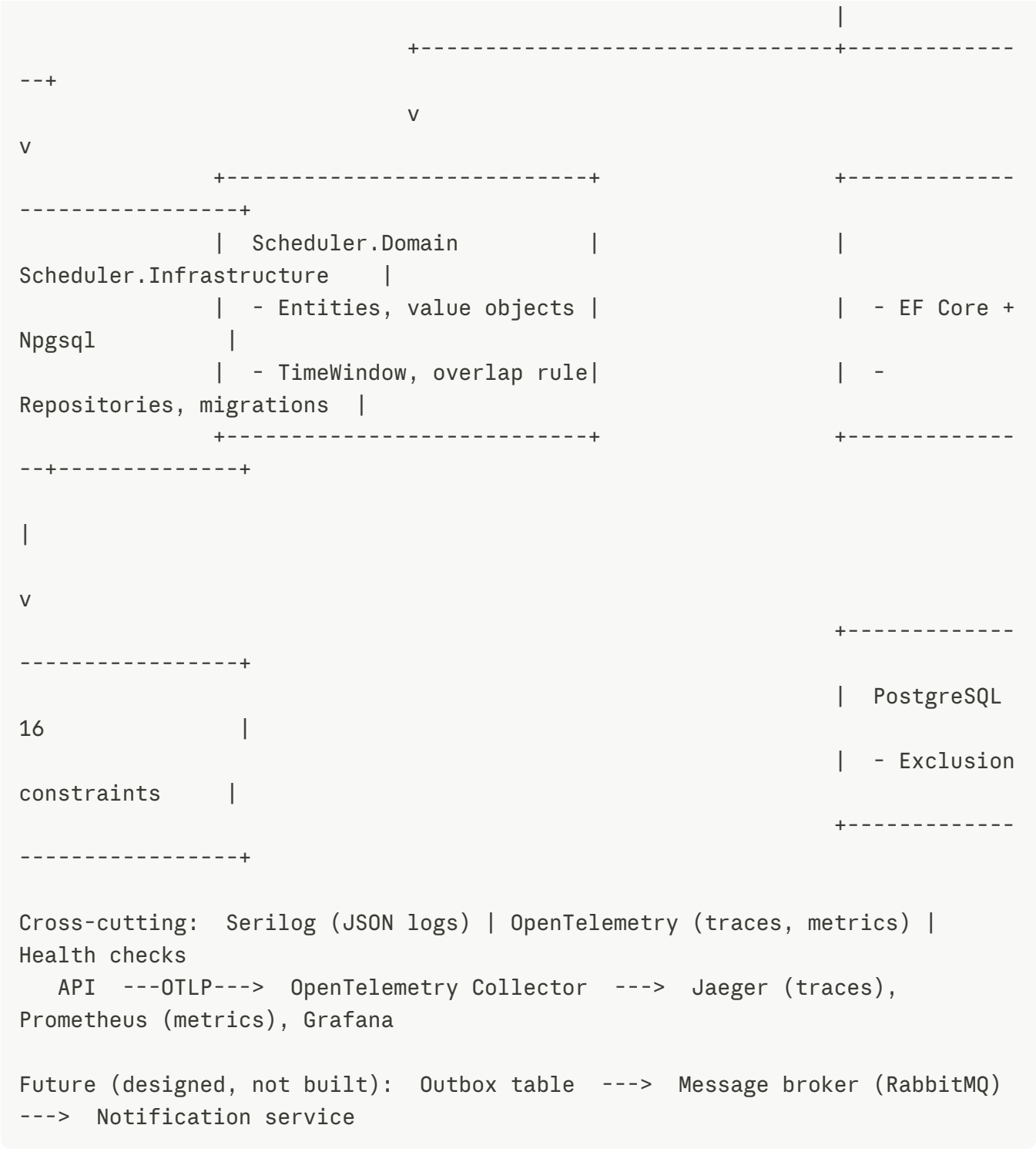
The brief leaves some points open. These are my reasonable assumptions.

1. Each ServiceType has a fixed duration in minutes (for example Oil Change 60, Brake Service 120). Duration is not chosen by the customer.
2. Each Dealership has a time zone and opening hours (default 08:00 to 18:00, Monday to Saturday). An appointment must start and finish inside opening hours.
3. All timestamps are stored and exchanged in UTC (ISO 8601). Opening hours are evaluated in the dealership's local time zone.

4. Start times fall on 15-minute boundaries. There is no buffer time between appointments.
5. A Technician is qualified for a ServiceType if a qualification record links them. Technicians and bays belong to exactly one dealership.
6. Any active bay at the dealership can host any service type. A future bay-type rule is possible (see section 13).
7. A vehicle must belong to the requesting customer, and the same vehicle cannot have two overlapping appointments.
8. Authentication is stubbed. In production the customer id would come from a validated JWT claim, not from the request body.
9. Appointments cannot be booked in the past or more than 180 days ahead.
10. A cancelled appointment immediately frees its bay and technician.

3. Architecture




|
+----------+
v
v
+----------+ +----------
Scheduler.Domain		Scheduler.Infrastructure
- Entities, value objects		- EF Core + Npgsql
- TimeWindow, overlap rule		- Repositories, migrations
+----------+ +----------
|
v
+----------
| PostgreSQL 16 |
| - Exclusion constraints |
+----------
Cross-cutting: Serilog (JSON logs) | OpenTelemetry (traces, metrics) | Health checks
API ---OTLP---> OpenTelemetry Collector ---> Jaeger (traces), Prometheus (metrics), Grafana
Future (designed, not built): Outbox table ---> Message broker (RabbitMQ) ---> Notification service

4. Components and their roles

Component	Role
Scheduler.Api	HTTP surface. Routing, request validation, mapping errors to HTTP status codes (ProblemDetails), correlation id handling, Swagger/OpenAPI. Contains no business rules.
Scheduler.Application	Use cases. `BookingService` orchestrates validation, resource selection and persistence. `AvailabilityService` answers slot queries. Defines interfaces (ports) for storage and time.
Scheduler.Domain	Pure business model with no framework dependencies. Entities, the `TimeWindow` value object and the overlap rule. Easy to unit test.
Scheduler.Infrastructure	EF Core DbContext, repositories, migrations, the raw SQL for database constraints, and seed data.
PostgreSQL	System of record. Also the final arbiter of double-booking through exclusion constraints.
Observability stack	OpenTelemetry Collector, Jaeger, Prometheus and Grafana, started by docker compose as an optional profile.
Test projects	Unit tests (domain and application), integration tests (API plus a real PostgreSQL through Testcontainers).

5. Domain model

Entity	Key fields	Notes
Customer	Id, Name, Email, Phone	Owner of vehicles.
Vehicle	Id, CustomerId, Vin, Make, Model, Year	VIN is unique.

Entity	Key fields	Notes
Dealership	Id, Name, TimeZoneId, OpensAt, ClosesAt	Opening hours in local time.
ServiceType	Id, Name, DurationMinutes	Fixed duration per type.
ServiceBay	Id, DealershipId, Name, IsActive	Physical bay.
Technician	Id, DealershipId, Name, IsActive	Works at one dealership.
TechnicianQualification	TechnicianId, ServiceTypeId	Many-to-many link that defines qualified.
Appointment	Id, CustomerId, VehicleId, DealershipId, ServiceTypeId, TechnicianId, ServiceBayId, StartUtc, EndUtc, Status, CreatedUtc, IdempotencyKey	Status is Confirmed or Cancelled. EndUtc = StartUtc + service duration.

6. Data flow: booking an appointment

1. The client sends `POST /api/v1/appointments` with customerId, vehicleId, dealershipId, serviceTypeId and startUtc. It may send an `Idempotency-Key` header.
2. The API assigns or reads the `X-Correlation-Id`, starts a trace span and validates the request shape (FluentValidation). Invalid input returns `400`.
3. `BookingService` loads the service type, vehicle and dealership. Reference data is cached in memory with a short TTL. It rejects unknown ids (`404`) and a vehicle that does not belong to the customer (`422`).
4. It computes the window `[StartUtc, StartUtc + duration)` and checks it against opening hours, the 15-minute grid, and the past and future limits. Failures return `422`.
5. If an `Idempotency-Key` was already used, the original appointment is returned and nothing new is created.
6. A database transaction begins. The repository finds candidate bays (active, at this dealership, no confirmed overlap) and candidate technicians (active, qualified, at this dealership, no confirmed overlap). It picks the first bay by id and the technician with the lightest load that day.

7. If no bay or no technician is free, the API returns `409 Conflict` with a reason code (`NO_BAY_AVAILABLE`, `NO_TECHNICIAN_AVAILABLE`) and up to three suggested alternative start times.
8. Otherwise the Appointment row is inserted and the transaction commits.
9. If the insert hits an exclusion violation (SQLSTATE 23P01), another request won the race. The transaction rolls back and the service retries with fresh candidates, up to three times, then returns `409`.
10. The API returns `201 Created` with a `Location` header and the appointment body. Metrics and a structured log line record the outcome.

7. Availability and concurrency strategy

Overlap rule. Two windows `A` and `B` conflict when `A.start < B.end AND A.end > B.start`. Windows are half-open, so back-to-back appointments (one ends at 10:00, the next starts at 10:00) are allowed. This rule lives in `TimeWindow` in the Domain project and is the first thing covered by unit tests.

Why checking first is not enough. Two requests can both see a free bay and both insert. An application-level check alone has a race condition. So the database enforces the rule as well.

Defence in depth.

1. Application check inside a transaction gives fast, friendly errors and the suggested alternatives.
2. PostgreSQL exclusion constraints guarantee no overlap, whatever the application does. They apply only to confirmed appointments, so cancelling frees the slot.

```
CREATE EXTENSION IF NOT EXISTS btree_gist;

ALTER TABLE appointments ADD CONSTRAINT ex_bay_no_overlap
  EXCLUDE USING gist (service_bay_id WITH =,
                      tstzrange(start_utc, end_utc, '[)') WITH &&)
  WHERE (status = 'Confirmed');

ALTER TABLE appointments ADD CONSTRAINT ex_technician_no_overlap
  EXCLUDE USING gist (technician_id WITH =,
                      tstzrange(start_utc, end_utc, '[)') WITH &&)
  WHERE (status = 'Confirmed');

ALTER TABLE appointments ADD CONSTRAINT ex_vehicle_no_overlap
  EXCLUDE USING gist (vehicle_id WITH =,
                      tstzrange(start_utc, end_utc, '[)') WITH &&)
  WHERE (status = 'Confirmed');
```

These constraints are added in an EF Core migration with raw SQL, because EF Core cannot model them directly.

Contention is low by design. Each constraint is scoped to one resource, so two bookings for different bays or technicians never block each other. Only real conflicts collide.

8. API design

Versioned under `/api/v1`. Errors use RFC 7807 ProblemDetails with a stable `code` field.

Method and path	Purpose	Success	Failure
POST /appointments	Book an appointment	201 Created	400 invalid input, 404 unknown id, 409 no capacity, 422 rule violation
GET /appointments/{id}	Fetch one appointment	200	404
GET /customers/{id}/appointments	List a customer's appointments	200	404
POST /appointments/{id}/cancel	Cancel and free resources	204	404, 409 already cancelled

Method and path	Purpose	Success	Failure
GET /dealerships/{id}/availability?serviceTypeId=&date=	List open start times for a day	200	404
GET /health/live and /health/ready	Liveness and readiness	200	503

The OpenAPI document is generated from the code and published at `/swagger`. A `requests.http` file and a cURL script show the main flows and stand in for the stubbed client.

9. Technology choices

Choice	Why
C# on .NET 10 (current LTS)	Strong typing for a rules-heavy domain, good performance, first-class tooling, matches Keyloop's stack.
ASP.NET Core Web API	Mature, fast, built-in dependency injection, health checks and OpenTelemetry support.
EF Core with Npgsql	Productive data access and migrations. Raw SQL is still available for the exclusion constraints.
PostgreSQL 16	Range types and exclusion constraints let the database itself prevent double booking. Open source and easy to run in Docker.
FluentValidation	Keeps request validation readable and separate from business rules.
Serilog	Structured JSON logs with enrichment (correlation id, trace id).
OpenTelemetry	Vendor-neutral traces and metrics. Works with Jaeger, Prometheus, Grafana or any cloud backend.

Choice	Why
xUnit, FluentAssertions, Testcontainers	Fast unit tests, readable assertions, and integration tests against a real PostgreSQL instead of an in-memory fake that would hide constraint behaviour.
Swashbuckle (OpenAPI)	Live API contract that stands in for the stubbed frontend.
Docker and docker compose	Reproducible local setup for reviewers.
GitHub Actions	Build and test on every push.

10. Observability

Logging. Serilog writes structured JSON to stdout. Every request log carries `CorrelationId`, `TraceId`, `DealershipId` and the outcome. Personal data (email, phone) is never logged. Log levels: Information for outcomes, Warning for conflicts and retries, Error for unexpected failures.

Tracing. OpenTelemetry instruments ASP.NET Core, EF Core and Npgsql. A booking produces one trace with child spans for validation, availability query and insert, so slow steps are easy to see. The correlation id is returned in the `X-Correlation-Id` response header so support can find a trace.

Metrics (exposed for Prometheus):

- `scheduler_bookings_total{outcome}` where outcome is confirmed, conflict or rejected.
- `scheduler_booking_retries_total` for races lost to the database constraint.
- `scheduler_booking_duration_seconds` histogram.
- `scheduler_availability_query_duration_seconds` histogram.
- Standard HTTP server and .NET runtime metrics.

Health. `/health/live` checks the process. `/health/ready` also checks the database connection.

Alerts (suggested). High 5xx rate, booking p95 latency above a threshold, a sudden rise in conflict or retry ratio (a sign of capacity problems or lock contention).

11. Non-functional considerations

Scalability. The API is stateless and can run as many replicas behind a load balancer. The database is the only consistency point. Availability reads can move to read replicas. Large installations can partition appointments by dealership.

Performance. Composite indexes on (service_bay_id, start_utc, end_utc) and (technician_id, start_utc, end_utc). The gist constraints also provide indexed overlap lookups. Reference data is cached in memory. The booking path makes a small, fixed number of queries.

Reliability. Transactional booking, database-enforced invariants, bounded retries on constraint races, idempotency keys so client retries are safe, request timeouts, and health checks for orchestrators.

Maintainability. Layered structure with the domain free of framework code, interfaces at the boundaries, analyzers and `.editorconfig` for consistent style, and a CI pipeline that builds and runs all tests.

Security. Input validation, parameterised queries through EF Core, no personal data in logs, and an authentication hook (JWT bearer) ready to be enabled.

12. Testing strategy

- **Domain unit tests:** overlap rule (exact overlap, partial overlap, contained window, back-to-back allowed), opening-hours check, 15-minute grid, end time calculation.
- **Application tests:** books when a bay and a qualified technician are free, refuses when no bay is free, refuses when no technician is qualified, refuses when the only qualified technician is busy, picks a different technician when the first is busy, idempotent retry returns the same appointment, cancellation frees the slot.
- **Integration tests (Testcontainers PostgreSQL):** end-to-end booking through the HTTP API, and a concurrency test that fires many parallel requests for the same slot and asserts exactly one succeeds while the rest receive `409`.
- **Contract check:** the generated OpenAPI document is compared with the committed spec so the API contract cannot drift unnoticed.

13. Risks and future work

Item	Direction
Notifications (email or SMS on confirmation)	Add an outbox table written in the same transaction as the appointment, and a worker that publishes `AppointmentConfirmed` to a broker such as RabbitMQ. This avoids lost or phantom events.
Bay types (lift, alignment bay)	Add a required bay type to ServiceType and filter candidate bays by it.
Technician shifts, breaks and leave	Add a working-calendar table and exclude unavailable periods in candidate selection.
Smarter assignment	Optimise for skill match and load balancing, or run a periodic re-optimisation.
Multi-region growth	Partition by dealership or region and keep each dealership's data in one region.
Authentication and roles	Enable JWT bearer, derive customer id from claims, add staff roles for dealership users.

14. How GenAI was used in the design phase

> Edit this section so it describes what you actually did. Keep the structure, replace the bracketed text with your real examples. Reviewers assess how you directed, checked and owned the AI output.

Tools used: [name the tools, for example GitHub Copilot, Claude, Gemini CLI].

How I directed the AI. [Describe your approach. For example: I gave the AI the assessment brief and my chosen constraints (C#, PostgreSQL, backend only) and asked for options for the booking and concurrency design before choosing one.]

What the AI proposed and what I decided.

- [Example: the AI first proposed an application-level availability check only. I identified the race condition and added the PostgreSQL exclusion constraints as a second line of defence.]

- [Example: the AI suggested an in-memory database for tests. I replaced it with Testcontainers because exclusion constraints do not exist in in-memory providers.]
- [Add a third example of something you changed or rejected.]

How I verified the output. [Describe your checks: read every generated file, ran the tests, tried edge cases such as back-to-back slots and parallel requests, checked the SQL by running it against PostgreSQL.]

Ownership. All design decisions in this document are mine. The AI accelerated drafting and surfaced alternatives, and I validated each decision before accepting it.